Scam Shield

Technical Design & Architecture

A free, open-source web app that helps non-technical users — think parents and grandparents — decide whether a suspicious email or text is a scam. This document is the engineering blueprint: what the system does, why it's built the way it is, and how to extend it without breaking the privacy contract.

Version	v0.1.0 (MVP)
Status	Released
Repository	github.com/srivatp2-code/scam-shield
License	MIT
Author	Sri Pusarla
Bundle size	≈118 KB gzipped / ≈400 KB raw

Contents

1. Executive Summary

Scam Shield is a single-page web application that classifies arbitrary text or screenshots as **safe**, **suspicious**, or **scam**, and returns a grandparent-readable explanation. There is no backend: the user supplies their own Anthropic API key (BYOK), and the browser calls `api.anthropic.com` directly using the official SDK with `dangerouslyAllowBrowser: true`. Verdicts are produced by a single structured-JSON call to Claude, primed with a system prompt that injects a community-curated library of known scam patterns.

Three properties define the design and constrain every decision:

- **Privacy is structural.** Because there is no backend, no scan data can leak from a server we control — there isn't one.
- **Accessibility is non-negotiable.** 18 px base font, 44 px tap targets, 375 px mobile-first layout, plain-English copy aimed at someone in their 70s.
- **Extensibility is community-driven.** Adding a new scam pattern is a single JSON file plus a PR; the loader picks it up at build time via Vite's `import.meta.glob`.

2. Goals and Non-Goals

Goals

- Produce a single, unambiguous verdict (■ / ■ / ■) per message, with concrete red-flag citations.
- Work for text messages, emails, and image screenshots — same interface, one button.
- Be usable one-handed on a 375 px-wide phone with no horizontal scrolling.
- Ship as a static site that any CDN can host. Build command is `npm run build`; output is `dist/`.
- Allow the scam pattern library to grow through community pull requests without modifying app code.

Non-goals

- Not a replacement for security advice. The disclaimer is explicit in the README and UI.
- Not a server-side service. No backend, no database, no per-user accounts.
- Not an analytics product. Zero telemetry, zero third-party scripts.
- Not multi-tenant. Each browser is its own world; the API key and history are device-local.
- Not multilingual in v0.1.0 — English-only copy and prompt.

3. System Architecture

Scam Shield is a static React SPA. There are only three actors at runtime: the user's browser, the user's localStorage, and Anthropic's API. Every other component in the diagram below is source code that compiles into the browser bundle.

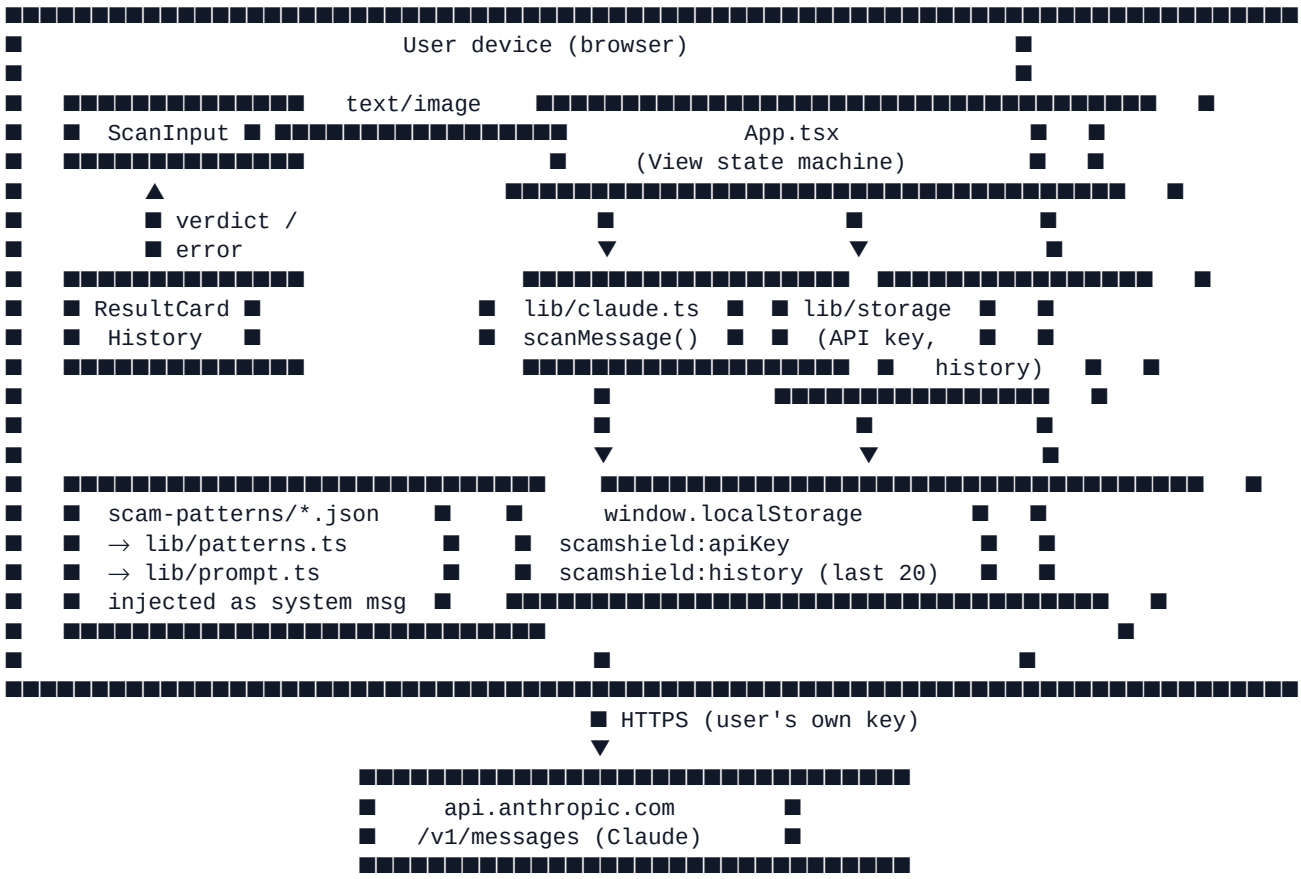

Figure 1. Runtime data flow. The dashed boundary is the device.

Layered view

The codebase is organized in three layers, with strict one-way dependencies:

- **UI layer** (`src/components/*` and `App.tsx`) — purely presentational. Owns the View state machine, handles input events, and renders results.
- **Lib layer** (`src/lib/*`) — business logic: API client, storage wrapper, prompt builder, pattern loader. No React imports.
- **Data layer** — TypeScript types in `src/types.ts` and JSON in `src/scam-patterns/`. No code.

UI imports lib; lib imports types and patterns; nothing imports UI. This means the entire scanning and storage system can be unit-tested without rendering React.

4. Technology Stack

Build	Vite 5 (rolldown) — dev server with HMR, production build to `dist/`.
UI	React 18 + TypeScript (strict mode).
Styling	Tailwind CSS v3 — no UI library, no CSS-in-JS, single utility class system.
Icons	lucide-react (tree-shakeable SVG components).
AI SDK	@anthropic-ai/sdk 0.97.1 with `dangerouslyAllowBrowser: true`.
State	React hooks only. No Redux, Zustand, etc. The app is too small.
Routing	None. Conditional render on a single View union.
Tests	Build-time TypeScript strict mode; manual smoke test via verification checklist.
Package mgr	npm (Node 18+).

Rationale for the absence of common things

- **No state management library.** The app has ~7 pieces of state; useState handles it.
- **No router.** One screen at a time, modeled as a View union; URL hash routing would add bytes for no value.
- **No CSS framework beyond Tailwind.** A component library would add 100 KB+ and impose its own accessibility opinions; we want explicit control over font sizes and tap targets.
- **No tests in v0.1.0.** Time-bound MVP. Phase 6 verification was manual via Claude Preview's MCP at 375 px. Adding Playwright is high-value future work.
- **No backend.** This is the load-bearing privacy claim; adding one defeats the architecture.

5. Data Model

All runtime types live in `src/types.ts`. The schema is intentionally small.

Verdict

```
type Verdict = "safe" | "suspicious" | "scam";
```

ScanResult — what Claude returns

```
interface ScanResult {
  verdict: Verdict;              // "safe" | "suspicious" | "scam"
  confidence: number;            // integer 1..10
  headline: string;              // one sentence, plain English
  red_flags: string[];           // specific phrases quoted from input
  what_to_do: string;            // grandparent-friendly next step
  if_already_clicked?: string;   // recovery — only set when verdict==="scam"
}
```

ScanHistoryItem — what gets persisted

```
interface ScanHistoryItem {
  id: string;                    // crypto.randomUUID(), with fallback
  timestamp: number;             // Date.now()
  input_preview: string;         // first 120 chars of text, or "[image]"
  result: ScanResult;
}
```

ScamPattern — community-contributed JSON

```
interface ScamPattern {
  id: string;                    // kebab-case, matches filename
  name: string;                  // short human label
  description: string;           // 1–2 sentences
  example: string;               // sanitized verbatim message
  red_flags: string[];           // 3–6 observable tells
  category:
    | "delivery" | "government" | "financial" | "romance"
    | "employment" | "tech-support" | "other";
}
```

Each scam pattern lives as a standalone JSON file under `src/scam-patterns/`. Vite inlines them at build time. The loader runs a runtime shape check so a malformed contribution is silently skipped (with a dev-mode warning) rather than crashing the app.

6. Component Design

6.1 Storage layer — src/lib/storage.ts

A thin `localStorage` wrapper with two namespaced keys (`scamshield:apiKey`, `scamshield:history`) and try/catch around every read and write. Private-browsing failures degrade silently: a user can still scan in an Incognito window, they just won't accumulate history.

History is capped at 20 entries, newest first. Cap is enforced on write — `addToHistory` prepends the new item then slices.

6.2 Pattern loader — src/lib/patterns.ts

Uses Vite's `import.meta.glob('../scam-patterns/*.json', { eager: true, import: 'default' })` to inline every pattern at build time. Each loaded object is validated against the ScamPattern shape via an `isScamPattern` type guard (id/name/description/example are strings, red_flags is a string array, category is one of the allowed enum values). `index.json` is skipped by path — it's a discovery aid, not a pattern.

6.3 Prompt builder — src/lib/prompt.ts

Single function: `buildSystemPrompt(patterns)`. Returns a static template with `{{PATTERNS}}` replaced by a formatted dump (name, category, description, example, indented red-flag bullets). The template itself lists the 12 scam pattern categories Claude should look for, the decision rules, and the strict JSON output contract. See Appendix A.

6.4 Claude API client — src/lib/claude.ts

Single public function: `scanMessage(input, apiKey, patterns)` → `Promise<ScanResult>`. Steps:

- Reject empty input early with a ScanError.
- Construct an Anthropic client with `dangerouslyAllowBrowser: true`.
- Choose model: `claude-sonnet-4-5` for image input, `claude-haiku-4-5` for text-only (faster, cheaper).
- Build a single user message: either `[{type:'text'}]` or `[{type:'image'}, {type:'text': 'Analyze this message screenshot.'}]`.
- Set `max_tokens: 1024` and the patterns-injected system prompt.
- Strip optional ```` ```json ```` code fences defensively, JSON.parse, then run an `isScanResult` shape check on the parsed object.
- Map any thrown error to a grandparent-friendly ScanError message via `formatApiError`.

6.5 UI components — src/components/*

Layout	Header with Shield wordmark + Settings gear. Centered `max-w-xl` main slot. Footer with GitHub link and privacy reminder.
PrivacyNote	■ banner shown on the main screen. Single sentence stating data flow.
ApiKeySetup	Three numbered steps (link → password input → save). Reused in both onboarding (initial) and settings (edit/clear) modes. Includes a 'Why do I need a key?' expander explaining BYOK.

ScanInput	Textarea + 'Or upload a screenshot' button. One input at a time — typing clears the image, uploading clears the text. Inline error display.
ResultCard	Colored verdict header (red/yellow/green), headline, red-flag bullets, highlighted 'What to do' panel, and a red-tinted 'If you already clicked' panel rendered only for scam verdicts.
HistoryList	Collapsible last-20. Each entry shows the verdict color dot, the input preview, and a relative timestamp ('1 min ago', 'Yesterday', 'May 12'). Click reopens the ResultCard. Clear-history link at the bottom.

7. Scam Classification Logic

Classification is single-call. We do not chain multiple model invocations, do not use embeddings, and do not maintain server-side state. Everything depends on the system prompt, the pattern library, and Claude's reasoning ability over a structured-JSON contract.

7.1 Inputs

- **Text mode:** raw user-pasted message, trimmed but otherwise untouched. Sent as a single text content block.
- **Image mode:** base64-encoded screenshot with explicit `media_type` (one of `image/jpeg | image/png | image/gif | image/webp`), followed by a fixed text prompt "Analyze this message screenshot."

7.2 Model selection

Text-only input	claude-haiku-4-5 — fast, cheap, capable enough for English scam detection.
Image input	claude-sonnet-4-5 — vision model with stronger OCR + reasoning over screenshots.
max_tokens	1024 — comfortably above typical structured output (~250–400 tokens).
Streaming	Not used. The UI shows 'Reading the message…' until the full JSON arrives, then validates and renders. Streaming partial JSON would complicate the parse/validate flow.

7.3 Prompt design — five mechanisms

- **Audience priming.** 'Imagine someone in their 70s' biases word choice toward plain English. Explicit ban on jargon: never say 'phishing,' say 'a fake message pretending to be USPS.'
- **Scam taxonomy.** 12 named patterns: urgency, authority impersonation, unusual payment, lookalike domains, credential requests, generic greetings, grammar mismatches, refund confirmations, wrong-number pivots, upfront-payment job offers, fake tech support, emotional manipulation.
- **Few-shot via pattern library.** Five seed patterns are injected verbatim. Each carries a sanitized example and 3–6 observable red flags. Community contributions expand this without code changes.
- **Decision rules.** Explicit tie-breakers — 'when in doubt between safe and suspicious, choose suspicious'; 'between suspicious and scam, require clear evidence to call it scam.' This nudges Claude toward calibrated outputs and prevents both crying-wolf and false-negatives equally.
- **Strict JSON contract.** The output schema is given inline. The model is instructed to return ONLY JSON, no markdown fences and no prose. The client strips fences defensively in case the model adds them anyway.

7.4 Output contract — what the model must produce

```
{
  "verdict": "safe" | "suspicious" | "scam",
  "confidence": <integer 1–10>,
  "headline": "<one sentence>",
  "red_flags": ["<phrase quoted from message>", ...],
  "what_to_do": "<concrete next step>",
  "if_already_clicked": "<recovery steps; OMIT unless verdict === 'scam'>"
}
```

If the response fails to parse or doesn't match this shape, the client throws a generic *Got an unexpected response* error rather than guessing.

7.5 Verdict semantics — how the UI interprets each value

verdict	Color	UI label	Renders recovery panel?
safe	green-600	■ Looks Safe	No
suspicious	yellow-500	■ Suspicious	No
scam	red-600	■ Scam	Yes (if `if_already_clicked` present)

7.6 Why a single-shot prompt instead of a classifier model?

- **No training data needed.** A fine-tuned classifier would require labeled examples and ongoing retraining as scams evolve. Claude already generalizes.
- **Reasoning is the product.** The user needs to see *why* something is a scam, not just a score. Generative output gives us free-form red flags and recovery advice that a classifier cannot.
- **Cost & latency are acceptable.** Haiku-class scans complete in roughly 1–2 seconds. The user is already in 'I'm worried about this message' mode; a brief wait is fine.
- **Updates ship without a model retrain.** Adding patterns is a PR. Tweaking decision rules is a prompt edit.

8. Privacy & Security Architecture

The privacy story is the architecture. There is no server with a hard drive that could be subpoenaed or breached, because there is no server. Concretely:

- **No backend.** The repo contains zero server code. The 'deploy' is uploading static files to a CDN.
- **BYOK.** The user supplies their own Anthropic key. The key is stored only in `localStorage`, never sent anywhere except as an Authorization header on direct calls to `api.anthropic.com`.
- **Zero third-party scripts.** No analytics, no error reporting, no fonts CDN, no CAPTCHA. The only outbound origin at runtime is `api.anthropic.com` — verified in Phase 6 via the Network panel.
- **Scan history is device-local.** Up to the last 20 results live in `localStorage`. Clearing browser data clears history. A Clear-history link gives an explicit in-app reset.
- **No PII collection.** The app never asks for a name, email, or phone number.

8.1 The `dangerouslyAllowBrowser` flag

The Anthropic SDK requires this opt-in flag for direct browser calls because, in a typical SaaS context, shipping an API key to the browser leaks it to whoever opens the page. In Scam Shield, the user IS the one shipping the key — to themselves. The flag is therefore not 'dangerous' in our deployment; it is the whole point of the BYOK design. **CLAUDE.md explicitly warns future contributors not to 'fix' this.**

8.2 Threat model

Within the deployment described above, the threats Scam Shield does and does not address:

- **Mitigated — server-side data breach.** No server, no breach.
- **Mitigated — third-party telemetry leakage.** No third parties.
- **Mitigated — credential exfiltration via supply chain.** The only key on the device is the user's own; no shared secrets.

- **Out of scope — browser-level malware.** A compromised browser can read localStorage. This is true of any local-storage application.
- **Out of scope — Anthropic's data retention.** Once the API call is in flight, Anthropic's usage policy governs it. Documented in the README disclaimer.
- **Out of scope — host CDN integrity.** A compromised static host could ship a malicious bundle. Mitigation: Subresource Integrity is a future addition.

9. UI State Machine & Accessibility

9.1 View states

```
type View =
  | "loading"      // mount: read API key + history, load patterns
  | "needs-key"    // first-run; show ApiKeySetup in onboarding mode
  | "idle"         // main screen: PrivacyNote + ScanInput + HistoryList
  | "scanning"     // ScanInput frozen, button reads "Reading the message…"
  | "result"       // ResultCard for the most recent scan
  | "settings";    // ApiKeySetup in edit/clear mode
```

9.2 Transitions

- **loading** → **needs-key**: no API key in storage.
- **loading** → **idle**: API key present.
- **needs-key** → **idle**: user saves a key (also reachable from settings).
- **idle** → **scanning**: user submits non-empty input.
- **scanning** → **result**: API returned a valid ScanResult; history is updated.
- **scanning** → **idle**: API or shape error; `scanError` is set and rendered inline by ScanInput.
- **result** → **idle**: user taps 'Check another message.'
- **idle / result** → **settings**: user taps the gear icon.
- **settings** → **idle**: cancel or save.
- **settings** → **needs-key**: 'Remove key from this device.'

9.3 Accessibility design

Base font	18 px set on <html>. All Tailwind utilities are recomputed against this root.
Tap targets	Every interactive element has `min-h-11` (= 2.75 rem ≈ 49.5 px). Verified at runtime.

Viewport	Mobile-first; tested at 375 × 812. Content uses `max-w-xl` centered with `px-5`. No horizontal scrolling at 375 px (verified).
Color contrast	All text on background pairs satisfy WCAG AA. Verdict header tiles use red-600/yellow-500/green-600 with appropriate foreground.
Focus states	Every button has `focus:outline-none focus:ring-2 focus:ring-gray-900` (or red-600 for destructive).
Live errors	Error containers use `role='alert'` so screen readers announce them.
Form labels	Every input has a real <label htmlFor> — no placeholder-as-label anti-patterns.
Language	Plain English. Read level < 8th grade in user-facing copy. No security jargon.

10. Error Handling and Resilience

Errors are rerouted through a single `ScanError` class whose message is always safe to render in the UI. Mappings are explicit:

Source	Message shown to user
HTTP 401	Your API key looks invalid. Double-check it in Settings.
HTTP 429	You're sending requests too fast. Wait a moment and try again.
HTTP 529	Anthropic's servers are busy. Try again in a few seconds.
APIConnectionError	Couldn't reach Anthropic. Check your internet connection.
JSON parse / shape failure	Got an unexpected response. Try again — if it keeps happening, open an issue.
Empty input	Please paste a message or upload a screenshot first.
Unsupported image type	That image type isn't supported. Try a JPEG, PNG, GIF, or WebP screenshot.

All localStorage operations are wrapped in try/catch. A storage failure (Safari private mode, quota exceeded, user disabled storage) degrades silently — the user can still scan, just won't persist history

between reloads.

11. Build, Bundle, and Deployment

Build pipeline

```
npm run build
  → tsc -b          (strict TypeScript across the project references)
  → vite build      (Rolldown bundler, Tailwind JIT, asset hashing)
  → dist/
      index.html                 0.6 KB  (entry)
      assets/index-*.css        12.0 KB  (Tailwind utilities)
      assets/index-*.js        365.0 KB  (React + SDK + app code)
      assets/node-*.js          15.2 KB  (browser shim for SDK node paths)
      assets/__vite-browser-external-*.js    0.1 KB

Total gzipped: ~118 KB. Target: under 500 KB. ✓
```

Why the "node-*.js" chunk exists

The Anthropic SDK ships an agent-toolset module that imports node-only builtins (`node:fs`, `node:path`, etc.). Vite externalizes these for the browser build and emits a tiny shim. The code paths that would call them are never executed in BYOK browser usage — only `client.messages.create` is. The shim is benign at runtime; the build-time warnings are cosmetic. Future cleanup: pin imports to `@anthropic-ai/sdk/index` or add a Vite optimizeDeps exclude.

Deployment

- **Cloudflare Pages**: connect GitHub repo, set build command `npm run build`, output directory `dist`, no env vars. Every push to `main` redeploys.
- **GitHub Pages, Netlify, Vercel, S3+CloudFront**: identical configuration. The bundle is fully static — no runtime requirements beyond a modern browser.
- **CSP & SRI** (future): a sensible Content-Security-Policy and Subresource Integrity on the JS bundle would harden against a host-level compromise.

12. Extensibility

Adding a new scam pattern

- Drop a new JSON file in `src/scam-patterns/`.
- Conform to the ScamPattern shape (Section 5). Filename matches `id`.
- Add a one-line entry to `src/scam-patterns/index.json` for discoverability.

- Open a PR. **Sanitize the example** — see docs/CONTRIBUTING.md for the full PII checklist (names, phone numbers, URLs in bracket notation, fake reference numbers).
- On merge, the next build picks it up automatically via `import.meta.glob`.

Tuning Claude's behavior

- Edit the template in `src/lib/prompt.ts`.
- Decision rules and the output schema are inline in the template — keep them in sync with `isScanResult` in `src/lib/claude.ts`.
- Bumping to a newer model is a two-line change at the top of `src/lib/claude.ts`.

Why not a plugin system?

Three things would justify one: more than ~50 patterns, contributors who need to write code rather than JSON, or runtime pattern loading from a remote source. None apply yet. A JSON-per-pattern + Vite glob is simpler and type-safe enough.

13. Limitations and Future Work

Known limitations of v0.1.0

- **English only.** Both the prompt and the user-facing copy assume English.
- **Claude can be wrong.** Both false positives and false negatives are possible. The README and the UI are explicit about this.
- **Single message at a time.** No conversation context; no carrying over what the user has scanned before. (Past scans live in history but do not feed the next prompt.)
- **No accessibility audit by a third party** — best effort against WCAG AA only.
- **Image upload size.** Not yet capped on the client; very large screenshots may slow the upload.
- **No Playwright / unit tests.** Smoke-tested manually in Phase 6.

Roadmap candidates (post-MVP)

- End-to-end browser tests in Playwright (golden-path + error paths).
- Localization: prompt + UI in at least Spanish and Mandarin.
- Optional pattern-feedback loop — a 'this verdict was wrong' link that opens a sanitized PR template.
- PWA install + offline cache for assets (still online for API call).
- Voice readout of the verdict for low-vision users.

- Image-size cap with friendly downscaling client-side.
- Subresource Integrity hashes on the production bundle.
- Pluggable model selection (let advanced users pick a different Claude model from Settings).

Appendix A — Full System Prompt

Below is the full prompt text built by `buildSystemPrompt()`. The literal `{{PATTERNS}}` token is replaced at runtime with each scam pattern formatted as a small block of name, category, description, example, and indented red-flag bullets.

```
You are a scam-detection expert helping a non-technical person —
imagine someone in their 70s — evaluate a suspicious message they received.

Your job: classify the message and explain your reasoning in plain English
that a grandparent would understand. Never use jargon. Never say "phishing" —
say "a fake message pretending to be USPS" or similar. Be specific by
quoting actual phrases from the message in your red_flags.

Look for these scam patterns:
- Urgency manipulation ("act now," "24 hours," "immediately," "final notice")
- Authority impersonation (banks, IRS, USPS, FedEx, Amazon, Microsoft, police)
- Unusual payment requests (gift cards, wire transfers, cryptocurrency, ...)
- Lookalike or suspicious domains (amaz0n.com, paypaI.com, IP addresses, ...)
- Credential requests (passwords, one-time codes, SSN, bank account numbers)
- Generic greetings ("Dear customer," "Dear user," "Hello sir/madam")
- Grammar or formatting that doesn't match the claimed sender
- Refund or payment confirmations the recipient didn't initiate
- "Wrong number" texts that pivot to friendship, romance, or crypto investing
- Job offers requiring upfront payment, training fees, or check-cashing
- Tech support claims of viruses or account compromise requiring remote access
- Emotional manipulation (fear, romance, sympathy, prize winnings)

Known scam patterns for reference:
{{PATTERNS}}

Decision rules:
- When in doubt between "safe" and "suspicious," choose "suspicious."
- When in doubt between "suspicious" and "scam," choose "scam" only if there
  is clear evidence (specific quoted red flags); otherwise "suspicious."
- Legitimate messages from real senders should be marked "safe."
- A legitimate-looking message that asks for sensitive information through an
  unusual channel is at least "suspicious."

Output: return ONLY a valid JSON object matching this shape. No markdown
fences, no preamble, no explanation outside the JSON.

{
  "verdict": "safe" | "suspicious" | "scam",
  "confidence": <integer 1-10>,
  "headline": "<one sentence summary in plain English>",
  "red_flags": ["<specific phrase or feature from the message>", ...],
  "what_to_do": "<grandparent-friendly action the recipient should take>",
  "if_already_clicked": "<recovery steps — include this field ONLY if
    verdict is 'scam', otherwise omit it entirely>"
}
```

Appendix B — ScanResult JSON Schema

Equivalent JSON Schema for the structured output. The runtime validator in `src/lib/claude.ts` implements this check by hand (no external schema library).

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "additionalProperties": false,
  "required": ["verdict", "confidence", "headline", "red_flags", "what_to_do"],
  "properties": {
    "verdict": { "type": "string", "enum": ["safe", "suspicious", "scam"] },
    "confidence": { "type": "integer", "minimum": 1, "maximum": 10 },
    "headline": { "type": "string", "minLength": 1 },
    "red_flags": {
      "type": "array",
      "items": { "type": "string" }
    },
    "what_to_do": { "type": "string", "minLength": 1 },
    "if_already_clicked": { "type": "string" }
  },
  "allOf": [
    {
      "if": { "properties": { "verdict": { "const": "scam" } } },
      "then": {}
    },
    {
      "if": { "not": { "properties": { "verdict": { "const": "scam" } } } },
      "then": { "not": { "required": ["if_already_clicked"] } }
    }
  ]
}
```

Appendix C — Project File Tree

```
scam-shield/
■■■ CLAUDE.md                project contract for future contributors
■■■ README.md                user-facing readme
■■■ LICENSE                  MIT
■■■ .gitignore
■■■ .env.example             intentionally empty (no env vars)
■■■ package.json
■■■ package-lock.json
■■■ vite.config.ts
■■■ tailwind.config.js
■■■ postcss.config.js
■■■ tsconfig.json
■■■ tsconfig.app.json
■■■ tsconfig.node.json
■■■ eslint.config.js
■■■ index.html               single entry point
■■■ public/
■   ■■■ icon.svg             shield favicon
■■■ src/
■   ■■■ main.tsx             React root bootstrap
■   ■■■ App.tsx              View state machine
■   ■■■ index.css            Tailwind directives + 18 px base
■   ■■■ types.ts             Verdict / ScanResult / ScanHistoryItem / ScamPattern
```

```
■   ■■■ components/
■   ■   ■■■ Layout.tsx
■   ■   ■■■ PrivacyNote.tsx
■   ■   ■■■ ApiKeySetup.tsx
■   ■   ■■■ ScanInput.tsx
■   ■   ■■■ ResultCard.tsx
■   ■   ■■■ HistoryList.tsx
■   ■■■ lib/
■   ■   ■■■ claude.ts        scanMessage() + ScanError
■   ■   ■■■ storage.ts       localStorage wrapper
■   ■   ■■■ prompt.ts        buildSystemPrompt(patterns)
■   ■   ■■■ patterns.ts      loadPatterns() via import.meta.glob
■   ■■■ scam-patterns/
■       ■■■ index.json       discovery list (skipped by loader)
■       ■■■ usps-package-fee.json
■       ■■■ irs-back-taxes.json
■       ■■■ wrong-number-crypto.json
■       ■■■ refund-confirmation.json
■       ■■■ job-offer-upfront.json
■■■ docs/
    ■■■ CONTRIBUTING.md      pattern-PR rules + PII sanitization
    ■■■ TECH_DESIGN.pdf      this document
```